



INTRODUCTION TO PROGRAMMING AND C LANGUAGE

GMU SAMBALPUR
Dept. Of Computer Science

What is Programming?

Definition

Programming is the process of **writing instructions** that a computer can understand and follow, in order to perform a **specific task**.

Simple Explanation

- Computers **do not think** like humans — they cannot guess what we mean.
- We must tell them **exactly what to do, in a specific order** — nothing can be skipped or assumed.
- Programming is essentially **talking to a computer**, using a language it understands.

Real-Life Analogy — The Chef

Computer = Chef

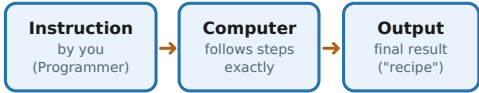
You (the programmer) give the chef a **recipe** (the program) to follow.

Clear Steps = Correct Output

If the steps are **clear and correct**, the chef cooks the meal exactly as you wanted.

⚠ If you **forget a step** — like telling the chef to turn on the stove — the chef (computer) gets **confused**. In programming, this confusion shows up as an **error**.

How This Looks as a Flow



Key Terms Building Up to "Programming"

Term	Meaning
Program	A set of instructions written for a computer to follow
Programming Language	A way (with rules and words) of writing those instructions so the computer can understand them

A programming language then branches into two broad categories — **High-Level** and **Low-Level** — covered on the next page.

Fun Tip

No stove step = no dinner ☹ — computers need **every single step** spelled out!

High-Level vs Low-Level Language

Definition

Every programming language falls broadly into one of two categories, based on **how close it is to human language** versus **how close it is to the computer's own machine code** (binary 0s and 1s).

High-Level vs Low-Level

Aspect	High-Level Language	Low-Level Language
Readability	Easy for humans to read & write (uses words & symbols)	Close to machine code — hard for humans to read
Portability	Portable — runs on many types of computers	Usually tied closely to specific hardware
Speed	Slightly slower (needs translation first)	Fast & efficient — very close to hardware
Examples	C, Python, Java	Machine Language, Assembly Language
Typical Use	Apps, websites, games	Operating systems, embedded systems

The Two Types of Low-Level Language

Type	Description
Machine Language	Only 0s and 1s — the computer's native, rawest form of instruction
Assembly Language	Uses short words as a slightly more readable stand-in for machine code (e.g. <code>MOV</code> , <code>ADD</code>)

Where Does C Fit In?

C is officially a **high-level language** (humans can read and write it comfortably), but it is famous for also giving **low-level access** to memory through pointers — this is why C is often called a **"middle-level" language** in casual conversation, sitting between the two worlds.

Uses of Popular Programming Languages

Language	Common Use
C	Systems programming, embedded devices
Python	Web development, AI, automation
Java	Android apps
JavaScript	Websites (frontend/backend)

Note

- Whatever language you write in, the computer **only ever executes machine code** in the end — high-level code always gets translated down to it.
- Choosing a language usually depends on the **task**: system-level speed needs (C), rapid app development (Python), or portability across devices (Java).

Fun Tip

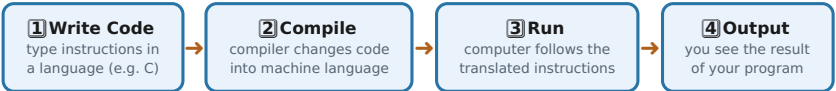
High-Level = "talks like you" 🗣️ | Low-Level = "talks like the machine" 🤖

⚙️ How a Program Runs

📖 Definition

Before we start writing C programs, it helps to understand the **journey** your code takes — from the moment you type it, to the moment you see a result on screen. This journey has **4 main stages**.

✂️ The 4 Stages



🔍 What Each Stage Actually Means

Stage	What Happens
Write Code	You type your instructions in a programming language, following its rules (syntax)
Compile	A program called a compiler checks your code for errors and translates it into machine language the computer can execute
Run	The computer's processor executes the translated instructions, one by one
Output	The final result is displayed to you — text, numbers, or any result your program was designed to produce

💡 **Compiler = Translator between you and the computer.** Just like you might need a translator to talk to someone who speaks a different language, the compiler translates **your C code** into the **0s and 1s** the computer's hardware actually understands.

🔗 Where Does This Fit With C Specifically?

In C, this process has a more precise name: **Compilation**. When you write a `.c` file and compile it, the compiler (like `gcc`) converts your human-readable code into an **executable file** that your operating system can run directly.

📌 Note

- If there is a **mistake in your code** (a syntax error), the compiler will **stop** at the "Compile" stage and show you an error message — the program won't even reach the "Run" stage.
- This is different from languages like Python, which are **interpreted** line-by-line instead of compiled all at once — but that's a topic for another day!

🌟 Fun Tip

Write → Compile → Run → Output — think of it as **Recipe → Kitchen → Cooking → Meal!** 🍴

C Introduction to C Language

Definition

C is a **high-level, general-purpose** programming language developed by **Dennis Ritchie** in **1972** at Bell Labs for the **UNIX operating system**. It is powerful, efficient, and widely used for both **system** and **application programming**.



Why Learn C?

- Forms the **foundation** for many other languages (C++, Java, Python all borrow heavily from C's syntax).
- Teaches important core concepts: **variables, loops, functions, and memory management**.
- Helps you understand **how computers actually work internally** — closer to the hardware than most modern languages.

Where Is C Used?

Area	Example
Operating Systems	Parts of UNIX/Linux, and even parts of Windows
Embedded Systems	Software running inside everyday hardware devices
Microcontroller Programming	Programming small chips (e.g. in appliances, sensors)
Game Development	Performance-critical game engines and components

5 Key Features of C

#	Feature	What It Means
1	Structured Language	Organizes code into small, manageable parts (functions/blocks)
2	Fast & Efficient	Runs quickly because it stays close to the hardware
3	Portable	Works on different computers with little to no change
4	Low-Level Access	Can directly work with memory using pointers
5	Rich Library	Comes with many built-in functions, and you can create your own too

Fun Fact: Even though C is over **50 years old**, it is still widely used today in core systems and high-performance applications — very few technologies from 1972 are still this relevant!

Fun Tip

C is the **"grandparent" language** 🦋 — C++, Java, and even parts of Python owe their syntax to it!

Your First C Program — "Hello, World!"

Why "Hello, World!"?

By tradition, the **very first program** anyone writes in a new language simply displays the text "Hello, World!" on the screen. It's small enough to focus on **understanding the structure** of the language, rather than solving a complex problem.

The Program

```
#include <stdio.h>

int main()
{
    printf("Hello, World!\n");
    return 0;
}
```

Output

```
Hello, World!
```

Breaking Down Every Line

Line / Symbol	Meaning
#include <stdio.h>	Adds the Standard Input/Output library , so we can use functions like <code>printf()</code>
int main()	The starting point of every C program — execution always begins here
{ }	Curly braces mark the beginning and end of the function's block of statements
printf("Hello, World!\n")	Prints text to the screen; <code>\n</code> moves to a new line afterward
;	Marks the end of a statement — almost every line in C needs one
return 0;	Tells the operating system the program finished successfully

Visualizing the Structure

```
#include <stdio.h>
int main()
{
    printf(...);
    return 0;
}
```

← bring in the tools we need

← "start here" signpost

← beginning of the function

← the actual instruction

← "I'm done, and it worked"

← end of the function

Note

- Every C program **must have exactly one `main()` function** — it's the starting point the operating system looks for.
- Forgetting the semicolon `;` at the end of a statement is one of the **most common beginner errors** — the compiler will refuse to compile until it's fixed.
- `\n` inside a string is called an **escape sequence** — it represents a "new line," not the two characters `\` and `n`.

 **Fun Tip**

Every programmer's journey – in **every** language – starts with saying "Hello, World!" 🌟